



```

        board[i][j] = ' '
        min_eval = min(min_eval, eval)
    return min_eval

def get_best_move(board):
    best_move = None
    best_eval = float('-inf')
    for i in range(3):
        for j in range(3):
            if board[i][j] == ' ':
                board[i][j] = 'O'
                eval = minimax(board, 0, False)
                board[i][j] = ' '
                if eval > best_eval:
                    best_eval = eval
                    best_move = (i, j)
    return best_move

def play_game():
    board = [[' ' for _ in range(3)] for _ in range(3)]
    current_player = 'X'

    while not is_game_over(board):
        print_board(board)

        if current_player == 'X':
            row, col = map(int, input("Enter your move (row and column separated by space): ").split())
            if board[row][col] == ' ':
                board[row][col] = 'X'
                current_player = 'O'
            else:
                print("Invalid move. Try again.")
        else:
            print("AI is thinking...")
            move = get_best_move(board)
            board[move[0]][move[1]] = 'O'
            current_player = 'X'

    print_board(board)

    if is_winner(board, 'X'):
        print("You win!")
    elif is_winner(board, 'O'):
        print("AI wins!")
    else:
        print("It's a draw!")

if __name__ == "__main__":
    play_game()

```

## Alpha Beta Pruning

MAX,MIN=1000,-1000

```
def minmax(depth,nodeIndex,maximizing_player,values,alpha,beta):
    if depth==3:
        return values[nodeIndex]
    if maximizing_player:
        best=MIN
        for i in range(0,2):
            val=minmax(depth+1,nodeIndex*2+i,False,values,alpha,beta)
            best=max(best,val)
            alpha=max(alpha,best)
            if beta<=alpha:
                break
        return best
    else:
        best=MAX
        for i in range(0,2):
            val=minmax(depth+1,nodeIndex*2+i,True,values,alpha,beta)
            best=min(best,val)
            beta=min(beta,best)
            if beta<=alpha:
                break
        return best

if __name__=='__main__':
    values=[3, 5, 6, 9, 1, 2, 0, -1]
    print("The optimal value is ",minmax(0,0,True,values,MIN,MAX))
```

## 8 Puzzle

hooray="hooorrrayyy"

```
def print_state(state):
    for row in state:
        print(" ".join(map(str, row)))
```

```
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
```

```

def move_up(state):
    i, j = find_blank(state)
    if i > 0:
        new_state = [row[:] for row in state]
        new_state[i][j], new_state[i-1][j] = new_state[i-1][j], new_state[i][j]
        return new_state
    else:
        return None

def move_down(state):
    i, j = find_blank(state)
    if i < 2:
        new_state = [row[:] for row in state]
        new_state[i][j], new_state[i+1][j] = new_state[i+1][j], new_state[i][j]
        return new_state
    else:
        return None

def move_left(state):
    i, j = find_blank(state)
    if j > 0:
        new_state = [row[:] for row in state]
        new_state[i][j], new_state[i][j-1] = new_state[i][j-1], new_state[i][j]
        return new_state
    else:
        return None

def move_right(state):
    i, j = find_blank(state)
    if j < 2:
        new_state = [row[:] for row in state]
        new_state[i][j], new_state[i][j+1] = new_state[i][j+1], new_state[i][j]
        return new_state
    else:
        return None

def calculate_heuristic(state, goal_state):
    # Simple heuristic: count the number of misplaced tiles
    h = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != goal_state[i][j]:
                h += 1
    return h

```

```

def a_star(initial_state, goal_state):
    OPEN = [(calculate_heuristic(initial_state, goal_state), 0, initial_state)]
    CLOSED = set()

    while OPEN:
        f, g, current_state = min(OPEN)
        OPEN.remove((f, g, current_state))
        CLOSED.add(tuple(map(tuple, current_state)))

        print_state(current_state)
        print(" ")
        if current_state == goal_state:
            print(f"{hooray}Solution found!")
            return

        successors = [
            (move_up(current_state), "UP"),
            (move_down(current_state), "DOWN"),
            (move_left(current_state), "LEFT"),
            (move_right(current_state), "RIGHT")
        ]

        successors = [(s, move) for s, move in successors if s is not None and tuple(
            map(tuple, s)) not in CLOSED]

        for successor, move in successors:
            h = calculate_heuristic(successor, goal_state)
            g_successor = g + 1
            f_successor = g_successor + h

            if (h, g_successor, successor) not in OPEN:
                OPEN.append((f_successor, g_successor, successor))

    print(f"{hooray}No solution found.")

# Updated example usage with the provided input
initial_state = [
    [1, 2, 3],
    [8, 0, 4],
    [7, 6, 5]
]
goal_state = [
    [2, 8, 1],
    [0, 4, 3],
    [7, 6, 5]
]
a_star(initial_state, goal_state)

```

## Hill Climb

```
def hill_climbing(func, start, step=0.01, max_iter=1000):
    x = start
    for _ in range(max_iter):
        fx = func(x)
        fx_positive = func(x + step)
        fx_negative = func(x - step)

        if fx_positive > fx and fx_positive >= fx_negative:
            x += step
        elif fx_negative > fx and fx_negative > fx_positive:
            x -= step
        else:
            break
    return x, func(x)

if __name__ == "__main__":
    while True:
        try:
            func_str = input("Enter a function of x (e.g., -(x-2)**2 + 4): ")
            func = eval(f"lambda x: {func_str}") # Convert string to function
            func(0) # Test the function with x = 0
            break
        except Exception as e:
            print(f"Invalid function. Please try again. Error: {e}")

    while True:
        try:
            start = float(input("Enter the starting value (e.g., 0): "))
            break
        except ValueError:
            print("Invalid input. Please enter a valid number.")

    max_x, max_val = hill_climbing(func, start)
    print(f"Maxima found at x = {max_x}")
    print(f"Maximum value = {max_val}")
```

## Logistic Regression

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris=datasets.load_iris()
x = iris.data[:, :2]
y=(iris.target!=0)*1
```

```
sc=StandardScaler()
x=sc.fit_transform(x)
print(x[:5])
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.4,random_state=42)
```

```
class LogisticRegression():
    def __init__(self):
        self.weights=None

    def sigmoid(self,z):
        return 1.0/(1.0+np.exp(-z))

    def cost(self,h,y):
        return -(y*np.log(h)+(1-y)*np.log(1-h))

    def gradient(self,x,h,y):
        return np.dot(x.T,(h-y))/y.shape[0]

    def fit(self,x,y,num_ iterations=200,learning_rate=0.01):
        self.weights=np.zeros(x.shape[1])
        for i in range(num_ iterations):
            z=np.dot(x,self.weights)
            h=self.sigmoid(z)
            grad=self.gradient(x,h,y)
            self.weights=self.weights-learning_rate*grad

    def predict(self,x):
        if self.weights is None:
            print("Train first")
            return None
        z=np.dot(x,self.weights)
        return (self.sigmoid(z)>0.5)*1
```

```
lr=LogisticRegression()
lr.fit(x_train,y_train,learning_rate=0.01,num_ iterations=200)
```

```
y_pred=lr.predict(x_test)
np.mean(y_pred==y_test)
print(np.mean(y_pred==y_test))
```

```
import matplotlib.pyplot as plt
plt.scatter(x_train[:, 0], x_train[:, 1], c=y_train)
```

```
plt.title('Logistic Regression Decision Boundaries')
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
```

## Naive Bayes

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load iris dataset
iris = load_iris()
X, y = iris.data, iris.target
class_names = iris.target_names

class NaiveBayes:
    def fit(self, X, y):
        self._classes = np.unique(y)
        self._mean = np.array([X[y == c].mean(axis=0) for c in self._classes])
        self._var = np.array([X[y == c].var(axis=0) for c in self._classes])
        self._priors = np.array([X[y == c].shape[0] / len(y) for c in self._classes])

    def predict(self, X):
        return np.array([self._predict(x) for x in X])

    def _predict(self, x):
        posteriors = [np.log(prior) + np.sum(np.log(self._pdf(idx, x)))
                       for idx, prior in enumerate(self._priors)]
        return self._classes[np.argmax(posteriors)]

    def _pdf(self, class_idx, x):
        mean, var = self._mean[class_idx], self._var[class_idx]
        numerator = np.exp(-(x - mean)**2 / (2 * var))
        denominator = np.sqrt(2 * np.pi * var)
        return numerator / denominator

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

# Create and train the Naive Bayes model
nb = NaiveBayes()
nb.fit(X_train, y_train)

# Make predictions
y_pred = nb.predict(X_test)
print('Accuracy: %.4f' % np.mean(y_pred == y_test))
# Print class names instead of class numbers
print("Predictions:", iris.target_names[y_pred])
```

### Optional confusion matrix

```
from sklearn.metrics import confusion_matrix, classification_report
# Print confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))
```

## KNN

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from collections import Counter

# Load data
X, y = load_iris(return_X_y=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X = X
        self.y = y

    def predict(self, X_test):
        predictions = []
        for x in X_test:
            # Compute distances
            distances = [np.linalg.norm(x - x_train) for x_train in self.X]
            # Get k nearest labels
            k_labels = [self.y[i] for i in np.argsort(distances)[:self.k]]
            # Majority vote
            predictions.append(Counter(k_labels).most_common(1)[0][0])
        return np.array(predictions)

# Train and predict
knn = KNN(k=3)
knn.fit(X_train, y_train)
```

```
y_pred = knn.predict(X_test)

# Accuracy
print("Accuracy:", np.mean(y_pred == y_test))
```

## **K-Means**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()
X = iris.data # Features (sepal length, sepal width, petal length, petal width)

def kmeans(X, k):
    centroids = X[np.random.choice(X.shape[0], k, replace=False)]

    for _ in range(100):
        distances = np.linalg.norm(X[:, None] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)
        centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

    return centroids, labels

# Apply custom k-means clustering
k = 3
centroids, labels = kmeans(X, k)

# Define colors for each cluster
colors = ['r', 'g', 'b']

# Plot the original data points with different colors for each cluster
for i in range(k):
    plt.scatter(X[labels == i, 0], X[labels == i, 1], c=colors[i], label=f'Cluster {i+1}')

# Plot the final cluster centroids
plt.scatter(centroids[:, 0], centroids[:, 1], marker='x', c='black', label='Centroids')

plt.title('K-Means Clustering on Iris Dataset')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.legend()
plt.show()
```